Объектно-ориентированное программирование

6 семестр
Лекция 1.

Содержание лекции:

- Классификация ПО по среде исполнения
- Клиент-серверное взаимодействие
- Проблема сложности и роль архитектуры
- Фундаментальные принципы проектирования

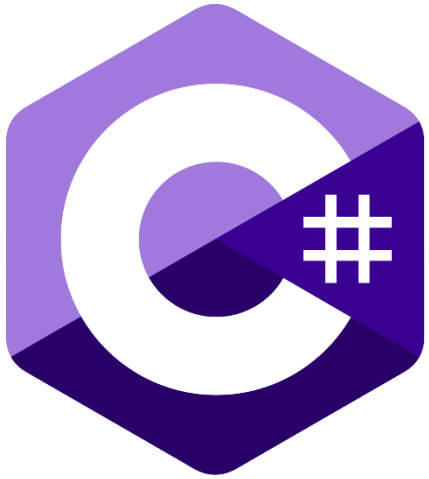
Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Десктоп (Desktop)

Теряющее популярность, но все еще очень активное направление — это десктоп. Это так называемые "толстые клиенты", — когда почти вся логика и обработка данных происходит на компьютере пользователя.

В этом направлении всегда были популярны такие языки, как **C++**, **Java** и **C#**, но на пятки признанным авторитетам наступают те языки, которые еще не так давно никто не планировал использовать для написания толстых клиентов. (**Java Script**).

На C# удобно писать под Windows: **Windows Forms, WPF.**



Языки в Web

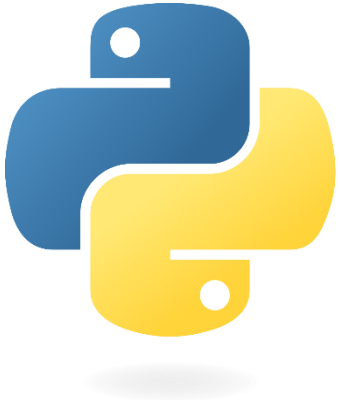
Веб делится на 2 подраздела: **бэкенд** и **фронтенд**. Любое веб-приложение в конечном итоге имеет ту часть, которая взаимодействует с пользователем, и почти всегда это **тонкий клиент (фронтенд)** — браузер, в котором крутятся приложения. Если раньше это был просто **HTML**, и вся логика происходила на сервере, то сейчас логики в браузере становится все больше и больше, и он становится по-настоящему "толстым" и "тяжелым".

HTML CSS JavaScript

Backend

PHP, Ruby, Python, JavaScript (Node), Perl

Go, C#, Elixir, Rust, Java, Clojure, Kotlin, Scala



Бэкендеры ответственны за серверную часть, то есть за то, что обеспечивает работу сервиса — взаимодействие с базами данных, обработку информации и вычисления.

Мобильная разработка

Мобильная разработка: Нативная и Кроссплатформенная

Нативная разработка (под конкретную ОС):

Android: Kotlin (современный стандарт), Java.

iOS: Swift (быстрый open-source язык), Objective-C (устаревает).

Кроссплатформенная разработка (один код для iOS и Android):

Flutter (Dart): Популярный фреймворк от Google.

React Native (JavaScript): Широко используемое решение от Meta.

Xamarin (C#): Позволяет создавать приложения для iOS, Android и Windows на базе .NET.

Серверное ПО

Системное и прикладное ПО

Системное ПО (работа с «железом»)

Что это: ОС, драйверы, управление ресурсами ПК.

Языки: C, C++, Rust.

Прикладное и серверное ПО

Что это: Программы для решения конкретных задач (аналитика, расчеты, базы данных).

Структура: Состоит из интерфейса пользователя и логики на сервере.

Языки: Java, Python, C#, C++ и практически любые другие.

Проблема сложности и роль архитектуры

Что это: Проверенные временем решения типовых проблем, возникающих при проектировании программного кода.

Универсальность: Паттерны не зависят от конкретного языка программирования. Принципы едины для C#, Java, Python и других ООП-языков.

Зачем использовать:

- **Готовые шаблоны:** Позволяют не «изобретать велосипед», а использовать стандартные способы формализации связей между объектами.
- **Чистота кода:** Упрощают структуру программы, делая её более гибкой.
- **Командная работа:** Создают «общий язык» для разработчиков. Зная паттерн, другой программист мгновенно понимает архитектурный замысел.

Фундаментальные принципы проектирования

1. Принцип KISS (Keep It Simple, Stupid)

Суть: Код должен быть максимально простым и ясным.

Главная цель паттернов: Упрощать структуру, а не усложнять её. Если применение паттерна делает код менее понятным — он не нужен.

2. Не злоупотребляйте паттернами («Паттерны ради паттернов»)

Избегайте избыточного проектирования (over-engineering).

Необоснованное использование шаблонов ведет к «раздуванию» кода и снижению его качества. Паттерн должен быть **оправданным** и эффективным способом решения конкретной задачи.

3. Принцип осознанности

Не используйте паттерн, если не понимаете его внутреннюю логику и механику. Копирование структуры без понимания принципа работы создает архитектурные долги.

4. Классификация решений

Паттернов много, и они решают разные задачи. Для удобства их разделяют на группы (порождающие, структурные, поведенческие) в зависимости от цели, которую они преследуют.